

1. Missing Destructor

Question:

What happens if we don't define a destructor in `MyVector`?

Answer:

- `data` is a dynamically allocated array (`new T[...]`).
- If we don't delete it in the destructor, memory is **never freed** → **memory leak**.
- The compiler generates a default destructor, which **does not delete dynamic memory**.

Solution:

```
~MyVector() {  
    delete[] data; // free allocated memory  
}
```

✓ Already in your code.

2. Copy Constructor / Deep Copy

Question:

What happens if we write:

```
MyVector<int> a;  
MyVector<int> b = a; // copy constructor
```

without a custom copy constructor?

Answer:

- Default copy constructor does **shallow copy**: `b.data` points to the same array as `a.data`.

- When **a** and **b** are destroyed, `delete[] data` is called twice → **undefined behavior** (crash).

Solution:

Implement a **deep copy constructor**:

```
MyVector(const MyVector& other)
    : data(new T[other._capacity]), _size(other._size),
    _capacity(other._capacity)
{
    for (size_t i = 0; i < _size; ++i) {
        data[i] = other.data[i]; // copy elements
    }
}
```

- Allocates **new memory** for the copy.
- Copies elements individually.
- Both vectors can now be destroyed independently.

3. Copy Assignment Operator

Question:

How do we handle:

```
MyVector<int> a;
MyVector<int> b;
b = a; // copy assignment
```

without breaking memory safety?

Answer:

- Default copy assignment does shallow copy → same problem as copy constructor.
- Must implement deep copy.

Solution:

```

MyVector& operator=(const MyVector& other) {
    if (this == &other) return *this; // handle self-assignment

    delete[] data;                // free old memory
    _size = other._size;
    _capacity = other._capacity;
    data = new T[_capacity];       // allocate new memory
    for (size_t i = 0; i < _size; ++i) {
        data[i] = other.data[i];   // copy elements
    }
    return *this;
}

```

4. Out-of-Range Checks

Question:

What if someone accesses `vec[10]` when `size = 5`?

Answer:

- `operator[]` in STL **does not check bounds**. Accessing out-of-bounds is **undefined behavior**.
- `at()` provides **bounds checking** and throws `std::out_of_range`.

Solution: Add `at()` method:

```

T& at(size_t index) {
    if (index >= _size) throw std::out_of_range("Index out of range");
    return data[index];
}

const T& at(size_t index) const {
    if (index >= _size) throw std::out_of_range("Index out of range");
    return data[index];
}

```

- Safer for production/debug code.
-

5. Memory Leak / Resize Handling

Question:

What if `push_back` throws while copying elements during `resize`?

Answer:

- In your simple version, if `T`'s copy constructor throws during `resize()`, old data may remain valid, but partially copied array is lost → memory leak.
- In interviews, they might ask **strong exception guarantee**.

Solution (simpler explanation):

- Allocate new memory.
- Copy elements safely.
- Only after successful copy, delete old memory.

```
void resize(size_t new_capacity) {
    T* new_data = new T[new_capacity]; // may throw
    for (size_t i = 0; i < _size; ++i) {
        new_data[i] = data[i];          // may throw
    }
    delete[] data;                      // delete old only after
success
    data = new_data;
    _capacity = new_capacity;
}
```

6. Example: Copy & Bounds in Action

```
MyVector<int> a;
a.push_back(10);
a.push_back(20);
```

```
MyVector<int> b = a; // uses copy constructor
b.push_back(30);

std::cout << "a size: " << a.size() << "\n"; // 2
std::cout << "b size: " << b.size() << "\n"; // 3

try {
    std::cout << a.at(5); // throws out_of_range
} catch (const std::out_of_range& e) {
    std::cout << "Exception caught: " << e.what() << "\n";
}
```

- Shows **copy constructor** works (deep copy).
- Shows **out-of-range check** works.